

2. Apply Window Function

1.RANK

SELECT

emp_id,

emp_name,

salary,

RANK() OVER(ORDER BY salary DESC) AS salary_rank

FROM Employee;

Result Grid			 Filter Rows:		E
	emp_id	emp_name	salary	salary_rank	
▶	106	Neha	90000.00	1	
	109	Rohan	85000.00	2	
	104	Sneha	80000.00	3	
	108	Pooja	72000.00	4	
	102	Kajal	70000.00	5	
	107	Vikas	65000.00	6	
	103	Amit	60000.00	7	
	110	Anjali	58000.00	8	
	105	Karan	55000.00	9	
	101	Pranali	50000.00	10	

2. Row Number

SELECT

emp_id,

emp_name,

dept_id,

salary,

ROW_NUMBER() OVER(PARTITION BY dept_id ORDER BY salary DESC) AS row_num

FROM Employee;

emp_id	emp_name	dept_id	salary	row_num
108	Pooja	1	72000.00	1
105	Karan	1	55000.00	2
101	Pranali	1	50000.00	3
106	Neha	2	90000.00	1
109	Rohan	2	85000.00	2
102	Kajal	2	70000.00	3
103	Amit	2	60000.00	4
104	Sneha	3	80000.00	1
107	Vikas	3	65000.00	2
110	Anjali	3	58000.00	3

3. DENSE_RANK()

```
SELECT  
    emp_id,  
    emp_name,  
    salary,  
    DENSE_RANK() OVER (ORDER BY salary DESC) AS dr  
FROM Employee;
```

emp_id	emp_name	salary	dr
106	Neha	90000.00	1
109	Rohan	85000.00	2
104	Sneha	80000.00	3
108	Pooja	72000.00	4
102	Kajal	70000.00	5
116	Rahul	68000.00	6
107	Vikas	65000.00	7
103	Amit	60000.00	8
110	Anjali	58000.00	9
105	Karan	55000.00	10
101	Pranali	50000.00	11

4. NTILE(4)

```
SELECT emp_id, emp_name, salary,  
NTILE(4) OVER(ORDER BY salary DESC) AS Quartile  
FROM Employee;
```

emp_id	emp_name	salary	Quartile
106	Neha	90000.00	1
109	Rohan	85000.00	1
104	Sneha	80000.00	1
108	Pooja	72000.00	2
102	Kajal	70000.00	2
116	Rahul	68000.00	2
107	Vikas	65000.00	3
103	Amit	60000.00	3
110	Anjali	58000.00	3
105	Karan	55000.00	4
101	Pranali	50000.00	4

5. LAG()

```
SELECT emp_name,  
salary,  
LAG(salary) OVER(ORDER BY salary) AS Previous_Salary  
FROM Employee;
```

emp_name	salary	Previous_Salary
Pranali	50000.00	NULL
Karan	55000.00	50000.00
Anjali	58000.00	55000.00
Amit	60000.00	58000.00
Vikas	65000.00	60000.00
Rahul	68000.00	65000.00
Kajal	70000.00	68000.00
Pooja	72000.00	70000.00
Sneha	80000.00	72000.00
Rohan	85000.00	80000.00
Neha	90000.00	85000.00

6. LEAD()

```
SELECT emp_name,  
salary,  
LEAD(salary) OVER(ORDER BY salary) AS Next_Salary  
FROM Employee;
```

emp_name	salary	Next_Salary
Pranali	50000.00	55000.00
Karan	55000.00	58000.00
Anjali	58000.00	60000.00
Amit	60000.00	65000.00
Vikas	65000.00	68000.00
Rahul	68000.00	70000.00
Kajal	70000.00	72000.00
Pooja	72000.00	80000.00
Sneha	80000.00	85000.00
Rohan	85000.00	90000.00
Neha	90000.00	NULL

7. FIRST_VALUE()

```
SELECT emp_name,  
salary,  
FIRST_VALUE(salary)  
OVER(ORDER BY salary DESC) AS Highest_Salary  
FROM Employee;
```

emp_name	salary	Highest_Salary
Neha	90000.00	90000.00
Rohan	85000.00	90000.00
Sneha	80000.00	90000.00
Pooja	72000.00	90000.00
Kajal	70000.00	90000.00
Rahul	68000.00	90000.00
Vikas	65000.00	90000.00
Amit	60000.00	90000.00
Anjali	58000.00	90000.00
Karan	55000.00	90000.00
Pranali	50000.00	90000.00

8. LAST_VALUE()

```
SELECT emp_name,  
salary,  
LAST_VALUE(salary)  
OVER  
(  
ORDER BY salary  
ROWS BETWEEN UNBOUNDED PRECEDING  
AND UNBOUNDED FOLLOWING  
)  
AS Last_Salary  
FROM Employee;
```

emp_name	salary	Last_Salary
Pranali	50000.00	90000.00
Karan	55000.00	90000.00
Anjali	58000.00	90000.00
Amit	60000.00	90000.00
Vikas	65000.00	90000.00
Rahul	68000.00	90000.00
Kajal	70000.00	90000.00
Pooja	72000.00	90000.00
Sneha	80000.00	90000.00
Rohan	85000.00	90000.00
Neha	90000.00	90000.00

9.CUME_DIST()

```
SELECT emp_name,  
salary,  
CUME_DIST() OVER(ORDER BY salary) AS Cumulative_Distribution  
FROM Employee;
```

emp_name	salary	Cumulative_Distribution
Pranali	50000.00	0.09090909090909091
Karan	55000.00	0.18181818181818182
Anjali	58000.00	0.2727272727272727
Amit	60000.00	0.36363636363636365
Vikas	65000.00	0.45454545454545453
Rahul	68000.00	0.5454545454545454
Kajal	70000.00	0.6363636363636364
Pooja	72000.00	0.7272727272727273
Sneha	80000.00	0.8181818181818182
Rohan	85000.00	0.9090909090909091
Neha	90000.00	1

3. Apply View on the data

// View for High Salary Employees

```
CREATE VIEW HighSalaryEmployees AS
```

```
SELECT *
```

```
FROM Employee
```

```
WHERE salary > 70000;
```

// to show table

```
SELECT * FROM HighSalaryEmployees;
```

OUTPUT:-

emp_id	emp_name	salary	hire_date	dept_id
104	Sneha	80000.00	2026-02-10	3
106	Neha	90000.00	2026-06-01	2
108	Pooja	72000.00	2026-04-25	1
109	Rohan	85000.00	2026-05-18	2

4. Apply Trigger on the data

//Audit Table

```
CREATE TABLE Employee_Log (  
    log_id INT AUTO_INCREMENT PRIMARY KEY,  
    emp_id INT,  
    action_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

//Trigger

```
DELIMITER $$  
  
CREATE TRIGGER trg_employee_insert  
AFTER INSERT ON Employee  
FOR EACH ROW  
BEGIN  
    INSERT INTO Employee_Log(emp_id)  
    VALUES (NEW.emp_id);  
END$$  
  
DELIMITER ;
```

//to test trigger

```
INSERT INTO Employee  
VALUES (116,'Rahul',68000,'2026-06-25',2);
```

//TO VIEW TABLE

```
SELECT * FROM Employee_Log;
```

OUTPUT:-

log_id	emp_id	action_date
1	116	2026-06-25 13:35:58

5. HANDLE CASE WHEN IN THE DATA

SELECT

emp_name,

salary,

CASE

WHEN salary >= 80000 THEN 'High Salary'

WHEN salary >= 60000 THEN 'Medium Salary'

ELSE 'Low Salary'

END AS Salary_Category

FROM Employee;

OUTPUT:-

emp_name	salary	Salary_Category
Pranali	50000.00	Low Salary
Kajal	70000.00	Medium Salary
Amit	60000.00	Medium Salary
Sneha	80000.00	High Salary
Karan	55000.00	Low Salary
Neha	90000.00	High Salary
Vikas	65000.00	Medium Salary
Pooja	72000.00	Medium Salary
Rohan	85000.00	High Salary
Anjali	58000.00	Low Salary
Rahul	68000.00	Medium Salary